📚 exbook 技術展示報告

> 公益性質去中心化書籍共享社群平台 — Django 5.2 全端實作展示

01 專案概述

exbook 是一套以公益為出發點的去中心化共享書籍平台，讓社群成員能夠自主上架、借閱、傳遞實體書籍。平台不設實體圖書館，書籍由用戶自行保管，透過面交完成所有流通。

後端架構

- Django 5.2 + Python 3.12+
- MariaDB 10.11
- django-fsm-2 3.x 狀態管理

前端與部署

- htmx 2.0.4 + Alpine.js 3.x
- Tailwind CSS（CDN + 自訂配置）
- Docker Compose（4 容器）+ Nginx

02 系統架構總覽

部署架構

- **客戶端：** 瀏覽器、Service Worker (Web Push)
- **Docker：** Nginx、Django Web、APScheduler、MariaDB
- **外部服務：** Google OAuth 2.0、Google Books API、SMTP

Django Apps

- `core/` : BaseModel、例外體系
- `accounts/` : 信用評分、驗證
- `books/` : SharedBook FSM
- `deals/` : Deal FSM、通知系統

03 技術亮點摘要

1. **抽象基礎模型**：UUID 主鍵、統一 timestamp
2. **有限狀態機**：`SharedBook`、`Deal`、`Appeal` 受 django-fsm-2 強保護
3. **服務層架構**：View → Service → Model 嚴格三層分離
4. **信用評分系統**：純函數計算 + 開根號邊際遞減公式
5. **自訂例外體系**：8 種語意化 `ServiceError` 例外類型
6. **身份驗證**：django-allauth + Google PKCE + 強制 Email 驗證
7. **ISBN 三層查詢**：本地 DB → Cache → Google Books API

04 抽象基礎模型

是什麼

- 全專案所有 Model 的共同基類
- 定義 UUID 主鍵與統一時間戳欄位
- 零重複代碼實現標準化

為什麼這樣設計

- UUID 避免自增 ID 可預測性安全問題
- 消除分散式環境主鍵衝突風險
- 統一時間戳，避免各 app 重複定義

我們怎麼用：`core/models.py`

```python
class BaseModel(models.Model):
    id = models.UUIDField(
        primary_key=True,
        default=uuid.uuid4,
        editable=False
    )
    created_at = models.DateTimeField(
        auto_now_add=True
    )
    class Meta:
        abstract = True

class UpdatableModel(BaseModel):
    updated_at = models.DateTimeField(
        auto_now=True
    )
    class Meta:
        abstract = True
```

05 有限狀態機（FSM）django-fsm-2

機制與防護

- `protected=True` 在 ORM 層面強制狀態流轉
- 非法狀態流轉在 save 時即拋出 `TransitionNotAllowed`

應用實例

- **SharedBook：** 8 個狀態（如可移轉、已被預約、借閱中）
- **Deal：** 5 個狀態與 5 種交易類型

程式碼範例

```
status = FSMField(
    default=SharedBookStatus.SUSPENDED,
    protected=True
)

@transition(
    field=status,
    source=SharedBookStatus.SUSPENDED,
    target=SharedBookStatus.TRANSFERABLE
)
def list_for_transfer(self): ...

@transition(
    field=status,
    source=SharedBookStatus.TRANSFERABLE,
    target=SharedBookStatus.RESERVED
)
def reserve(self): ...
```

06 服務層架構

View → Service → Model

- 所有業務邏輯集中於 `services/` 獨立模組
- View 不直接操作 Model，Model 不含業務規則
- `@transaction.atomic` 確保資料庫更新原子性

為什麼這樣設計

- View 層保持薄且可替換
- 宣告式化字典消除散落的條件判斷

各模組套件

- `accounts/services/`: trust_service, violation_service 等
- `books/services/`: isbn_service, book_set_service 等
- `deals/services/`: deal_transition_service, notification_service 等

07 信用評分系統

純函數信用計算

- 基於借閱歷史的量化信用評分系統
- 信用等級 0–3 對應 `BORROWING_LIMITS`，設定解耦

開根號邊際遞減

- `floor(sqrt(score))` 公式天然防刷分
- 早期交易對星級影響顯著，後期趨於穩定

核心公式（`trust_service.py`）

```
def compute_trust_score(metrics) -> int:
    return base_score + (deals * 10) - \
           (overdue * 10) + (avg_rating * 5)

def compute_trust_stars(score: int) -> int:
    return max(1, min(5,
        math.floor(math.sqrt(score))
    ))
```

08 自訂例外體系

語意化例外階層

- 以 `ServiceError` 為根節點的例外繼承樹
- 呼叫端精確捕捉特定業務錯誤類型
- View 層統一對應 HTTP 狀態碼

為什麼這樣設計

- 比裸露的 `Exception` 捕捉更清晰安全
- 新增 View 自動繼承錯誤處理語意

8 種語意化例外

- `ValidationError` (業務規則不符)
- `PermissionError` (403 無權限)
- `NotFoundError` (404 資源不存在)
- `ConflictError` (狀態衝突)
- `StateTransitionError` (FSM 非法)
- `ExternalServiceError` (API 失敗)
- `BusinessRuleError` (違反 BR-N)
- `IntegrityError` (完整性約束)

09 身份驗證與 10 ISBN 查詢

身份驗證 (OAuth + PKCE)

- `django-allauth` 65.x 實作
- 純 Email 登入 + Google OAuth 2.0
- 啟用 PKCE 防授權碼攔截，符合 OAuth 2.1
- 強制 Email 驗證 (mandatory)
- Signal 自動建立 `UserProfile`

ISBN 三層查詢策略

- **層級 1：**本地 DB 查詢 (零網路成本)
- **層級 2：**Django Cache (24h TTL)
- **層級 3：**Google Books API (httpx)
- 429 rate-limit 優雅降級，不崩潰
- 最大化本地命中率、最小化 API 呼叫

11 統一通知與 12 前端技術棧

三合一通知系統

- 單一 `notify()` 函數執行：DB 記錄、Web Push、Email
- 12+ 個具名通知函數 (如 `notify_deal_requested`)
- 即使 Push 授權失敗，DB 記錄依然保留

輕量前端技術棧

- **htmx 2.0.4**：全域 `hx-headers` 注入 CSRF，保留伺服器邏輯，無刷新互動
- **Alpine.js 3.x**：本地 UI 狀態管理
- **Tailwind CSS**：CDN + 自訂配置
- **PWA**：8 圖示、Service Worker、快取支援

🚀 為什麼選擇 PWA？(Engineering Value)

面交情境強化

- **離線存取**：面交地點可能在地下室或收訊不佳處。Service Worker 快取讓用戶仍可查看書籍詳情與面交碼。
- **Add to Home Screen**：消除瀏覽器網址列，提供原生 App 的沉浸感，提升用戶頻繁查看交易狀態的意願。

即時通訊成本優化

- **Web Push**：取代昂貴的 SMS。透過 VAPID 協議，伺服器能在用戶未開啟瀏覽器時，將借閱請求或到期提醒直接推送到手機通知欄。
- **低功耗**：相對於原生 App 常駐背景，PWA 透過瀏覽器代理通知，對用戶手機電量更友好。

⚡ htmx：捨棄 SPA 的策略性選擇

研發效率與維護性

- **Server-Side Truth**：所有 HTML 片段均由 Django 直接生成，不需同步前後端資料 Schema (JSON API)，減少 40% 以上的開發負擔。
- **SEO 友好**：核心頁面預設 SSR 渲染，非同步更新僅限局部區塊，確保搜尋引擎能完整爬取書籍資訊。

效能與體驗

- **極小 Payload**：htmx 函式庫僅 ~15KB (minified+gzipped)，遠小於 React/Vue bundle。
- **骨架屏 (Skeleton)**：透過 `_skeleton_book_card.html` 實作預加載預覽，消除資料非同步載入時的版面跳動 (Layout Shift)。

🔒 狀態機 (FSM) 如何保護業務價值？

數據完整性

- **防止非法跳躍**：例如「未經面交確認 (M) 絕不能直接跳轉至已完成 (D)」。FSM 在底層強制執行這些順序。
- **自動化 Side-effects**：狀態流轉後自動觸發 `notify_deal_meeted()` 等函數，確保業務行為與資料狀態高度一致。

審計與追蹤

- **單一入口點**：所有變更必須呼叫 `transition` 方法，這讓開發者能輕易在關鍵節點植入 Log 或審計追蹤，而不必搜尋全專案的 `.save()` 呼叫。

13 排程與 14 測試、15 匯出

Docker 排程與 DevOps

- **APScheduler 雙模式**：開發 Background / 生產 Blocking 獨立容器
- **CI/CD**：GitHub Actions (lint → test → deploy)

資料匯出限流

- JSON/CSV 匯出，Django Cache 限制每日 3 次
- `@transaction.atomic` 確保資料快照一致性

多層次測試架構

- **pytest**：核心框架，SQLite in-memory
- **factory-boy**：完整假資料套件，簡化 setup
- **Playwright**：E2E 測試 5 個完整業務流程
- 與 `pytest-playwright` 無縫整合

總結

- **架構紀律**：服務層、FSM 保護、統一例外體系三者合力，確保任何新增功能均在既有的業務規則框架內運作，不存在「繞過狀態機直接改資料」的捷徑。
- **輕量全端**：htmx + Alpine.js + Tailwind 的組合讓 Django Template 保有完整業務邏輯，同時實現 SPA 等級的無刷新互動體驗，完全不引入 Node.js 建置流程。
- **可驗證的工程品質**：`pytest` 單元測試 + `Playwright` E2E 測試 + `ruff` 靜態分析 + CI/CD 自動化部署，構成從代碼提交到生產上線的完整品質閘門。